

## Simulateur de Flotte de Drones

---

### But pédagogique

- **Objectif principal** : Vous familiariser avec le langage **Java**, la **programmation concurrente** (Threads) ainsi qu'à la production d'un programme **communiquant via le réseau**.
  - **Structure du projet** : Le projet comporte **deux parties** :
    - **Version sans communication réseau**
    - **Version avec communication réseau**
  - **Type de projet** : **Individuel ou par binôme**
- 

### 1. Description du Projet

Une entreprise de livraison souhaite utiliser une flotte de drones pour réaliser ses livraisons à domicile. La carte de la ville est représentée par une forme rectangulaire et elle contient des obstacles. Pour déplacer les drones d'un point à un autre sur cette carte, les ingénieurs de l'entreprise planifient un chemin à suivre par chaque drone. Chaque jour, les ingénieurs envoient aux drones des chemins codifiés pour effectuer leurs livraisons.

#### 1.1 Représentation de la Position d'un Drone

La position actuelle d'un drone est représentée par des coordonnées  $x$  et  $y$  de la carte ainsi qu'une lettre pour indiquer son orientation :

- La position  $0 \ 0 \ N$  indique que le drone est situé en haut à gauche de la carte et orienté vers le **nord**.
- Nous considérons qu'un drone est orienté vers le **nord** lorsqu'il se déplace sur le plateau de  $(x, y)$  à  $(x, y+1)$ .

#### 1.2 Codification des Mouvements

Le chemin envoyé pour chaque drone est codifié sous la forme d'une chaîne de caractères, où chaque caractère représente le mouvement à effectuer par le drone :

| Caractère | Action                                    |
|-----------|-------------------------------------------|
| <b>L</b>  | Tourner de 90° sur place à gauche         |
| <b>R</b>  | Tourner de 90° sur place à droite         |
| <b>M</b>  | Avancer d'une case dans la même direction |
| <b>B</b>  | Reculer d'une case dans la même direction |

### 1.3 Exemple de Déplacement

Suite à la réception de la chaîne `LMLM`, un drone positionné initialement à `2 5 N` effectue les manœuvres suivantes :

1. **L** : Tourner à gauche → Orientation : Ouest
2. **M** : Avancer d'une case → Position : `(1, 5)`
3. **L** : Tourner à gauche → Orientation : Sud
4. **M** : Avancer d'une case → Position : `(1, 4)`

**Position finale du drone :** `1 4 S`

### 1.4 Comportement du Drone face aux Obstacles

- Le drone **ne traverse pas les obstacles et ne les contourne pas**. Il exécute seulement la chaîne des commandes.
- Si la case suivante vers laquelle il doit avancer (commande `M`) ou reculer (commande `B`) est occupée par un obstacle, alors il **reste sur place** et continue à exécuter les commandes suivantes.
- L'exécution continue jusqu'à ce qu'une commande lui permet de se déplacer vers une case libre.
- **Comportement à la sortie de la carte** : Si un drone tente de sortir de la carte, il **continue son chemin dans la direction opposée** (wrapping).

### 1.5 Objectif du Projet

Programmer en **Java** un **simulateur des déplacements** d'un ensemble de drones sur un plateau avec des obstacles. Vous allez développer **3 versions** du simulateur de vol :

#### *Version 1 : Séquentielle*

Dans cette version, les drones se déplacent vers leurs destinations **l'un après l'autre** sans aucune synchronisation. Les positions des drones et leurs commandes sont fournis dans un **fichier texte**.

#### *Version 2 : Concurrente*

Dans cette version, les drones se déplacent vers leurs destinations **de façon concurrente** (simultanément). Les positions des drones et leurs commandes sont fournis dans un **fichier texte**. Chaque drone exécute ses mouvements dans un **Thread séparé**.

#### *Version 3 : Communicante*

Les drones se déplacent **de façon concurrente** et reçoivent leur position initiale et les commandes (chaîne de caractères) depuis le **réseau via un programme client** (station de commande). Dans cette version, vous devez également développer le **programme client** qui envoie les informations au programme serveur (simulateur) en utilisant les **Sockets Java**.

## 1.6 Architecture du Programme

Les 3 versions sont programmées dans le **même programme simulateur** et à son lancement, l'utilisateur choisit le mode à exécuter via un **menu** :

- Mode 1 : **Basique** (version séquentielle)
- Mode 2 : **Concurrent** (version concurrente avec threads)
- Mode 3 : **Serveur** (version communicante avec réseau)

## 1.7 Format du Fichier de Configuration

Les deux premières versions utilisent un **fichier texte** qui contient :

- **Première ligne** : Hauteur et largeur de la carte (séparées par un espace)
- **Lignes suivantes** : Coordonnées initiales de chaque drone, son orientation, et sa chaîne de mouvements

**Exemple** (pour 2 drones et un plateau de taille 20x20) :

```
20 20
0 5 N LMLMLMLMM
5 0 S MMRMMRMRRM
```

**Pour la version 3** (communicante) : Ces informations sont communiquées par le programme client au simulateur via une connexion socket.

---

## 2. Partie 1 : Version Basique du Simulateur

### 2.1 Fonctionnalités Requises

Dans cette version, il est attendu de programmer les fonctionnalités suivantes :

#### 2.1.1 Lecture du Fichier de Configuration

- Lire le fichier contenant la taille du plateau
- Lire les positions initiales de chaque drone
- Lire la chaîne de commandes pour chaque drone
- Stocker ces informations dans des structures de données appropriées (ArrayList, HashMap, etc.)

#### 2.1.2 Création et Affichage de la Carte

- Créer une représentation matricielle du plateau
- **Générer aléatoirement des obstacles** sur la carte
- Les obstacles sont désignés par le caractère #
- Afficher l'orientation actuelle du drone avec les caractères suivants :
  - > : Est
  - < : Ouest

- ^ : Nord
- v : Sud
- **Important** : Éviter de placer un obstacle sur la position initiale du drone

### 2.1.3 Déplacement des Drones Séquentiellement

- Les drones se déplacent **l'un après l'autre**
- Chaque drone **termine ses commandes** avant que le suivant ne commence
- **Affichage en continu** des positions de chaque drone après l'exécution de chaque commande
- Implémenter la logique de gestion des obstacles (reste sur place si mouvement bloqué)
- Implémenter le wrapping (continuer dans la direction opposée en cas de sortie de carte)

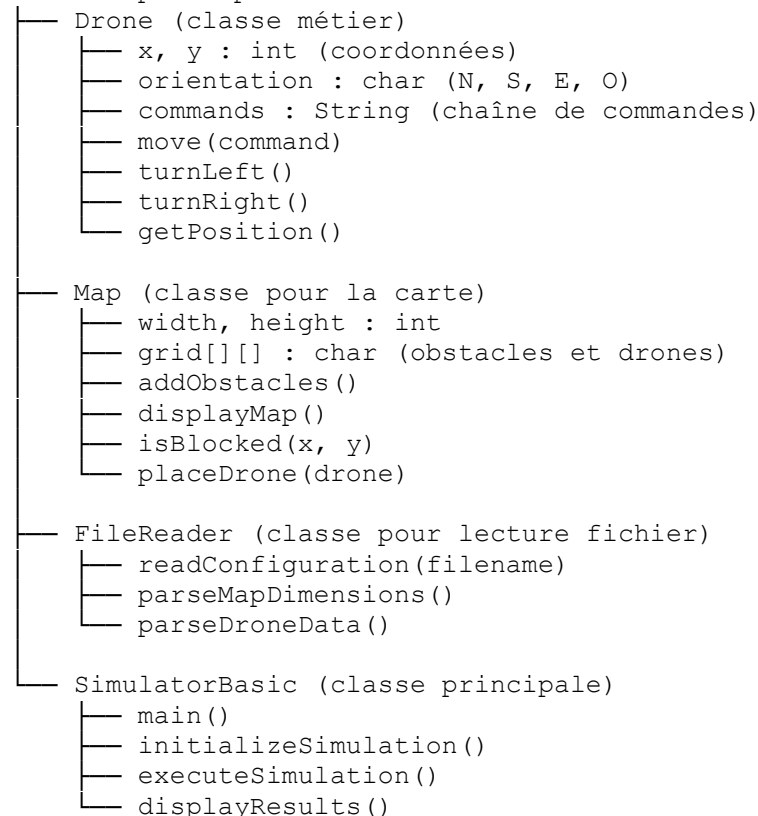
### 2.1.4 Affichage des Positions Finales

- Afficher clairement la position finale et l'orientation de chaque drone après l'exécution complète de ses commandes

## 2.2 Architecture Recommandée pour la Version Basique

Package: `com.drone.simulator`

Classes principales:



## 2.3 Exemple de Sortie Console

Résultats du Simulateur Basique  
=====

Initialisation de la carte (20x20)...

=====

Position du drone 0 : 4 5 W

• • •

```
position drone : 0 5 N
```

=====

=====

=====

Position du drone 1 : 5 2 S

• • •

Drone 0 : 1 3 S

```

Hauteur carte : 20 largeur carte : 20
position drone : 0 5 N
Commande : LMLMLMLMM

```

```

=====
  ## #   #
    # # #
      #   # #
##      #   #
#      #   # #
v #      #   ##
  #   #### #
# # #   # # #
# # #### #### # ##
      ##
    #       ##
      #   #
    # #   #
#      #   #
# #   #   #   #
  ## # #   ## #
#      #   # #
  # # # #   ##
    #       # ##
      #   # ##
=====

```

### 3. Partie 1 : Version Concurrente du Simulateur

#### 3.1 Modifications par rapport à la Version Basique

Dans cette version, vous gardez les mêmes fonctionnalités que la version précédente, **mais les drones exécutent leurs commandes en concurrence (simultanément)**.

#### 3.2 Fonctionnalités Spécifiques

##### 3.2.1 Threads d'Exécution des Drones

- **Chaque drone exécute ses commandes dans un Thread séparé** (implémentation de l'interface Runnable)
- Chaque drone a son propre thread qui exécute sa méthode `run()`
- Les threads s'exécutent **simultanément** (concurrence)

#### Implémentation recommandée :

```

public class DroneWorker extends Thread {
    private Drone drone;
    private SimulatorMap map;
    private Object displayLock;
    private Condition displayCondition;

    @Override
    public void run() {
        for (char command : drone.getCommands().toCharArray()) {
            drone.executeCommand(command, map);
            // Synchronisation avec le thread d'affichage
        }
    }
}

```

```

        synchronized (displayLock) {
            // Notifier le thread d'affichage
            displayCondition.signal();
        }
    }
}

```

### 3.2.2 Thread d'Affichage

- **Un thread dédié à l'affichage en continu** de la carte avec les drones et les obstacles
- **Une seule carte est affichée** et mise à jour en continu
- Utiliser la fonction `clearScreen()` (ou `System.out.flush()`) pour rafraîchir l'écran
- **À chaque fois qu'un drone termine l'exécution d'un mouvement**, il se synchronise avec le thread d'affichage

#### Implémentation recommandée :

```

public class DisplayWorker extends Thread {
    private SimulatorMap map;
    private List<Drone> drones;
    private volatile boolean running = true;

    @Override
    public void run() {
        while (running) {
            clearScreen();
            map.displayMap();
            try {
                Thread.sleep(100); // Rafraîchissement toutes les 100ms
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    private void clearScreen() {
        // Effacer l'écran
        System.out.print("\033[H\033[2J");
        System.out.flush();
    }
}

```

## 3.3 Synchronisation entre Threads

Utiliser les mécanismes de synchronisation Java :

- **synchronized** : Pour protéger l'accès concurrent à la carte
- **ReentrantLock et Condition** : Pour la synchronisation plus fine entre le thread d'affichage et les threads drone
- **CountDownLatch ou Barrier** : Pour attendre que tous les threads drones aient terminé

#### Exemple avec ReentrantLock et Condition :

```

ReentrantLock displayLock = new ReentrantLock();
Condition displayCondition = displayLock.newCondition();

```

```
// Dans DroneWorker.run()
for (char command : drone.getCommands().toCharArray()) {
    drone.executeCommand(command, map);

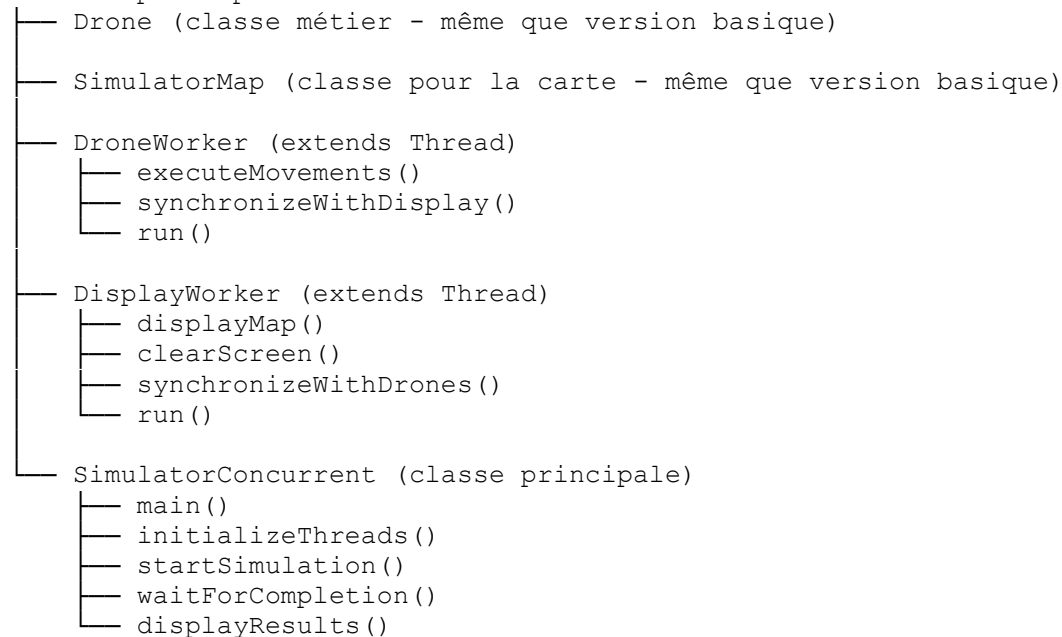
    displayLock.lock();
    try {
        displayCondition.signalAll(); // Réveiller le thread d'affichage
    } finally {
        displayLock.unlock();
    }
}

// Dans DisplayWorker.run()
displayLock.lock();
try {
    displayCondition.await(100, TimeUnit.MILLISECONDS);
    map.displayMap();
} finally {
    displayLock.unlock();
}
```

### 3.4 Architecture Recommandée

Package: com.drone.simulator.concurrent

Classes principales:



## 4. Partie 2 : Version Communicante du Simulateur

### 4.1 Description Générale

Cette version utilise les **mêmes fonctions que la version concurrente**, mais la **taille de la carte**, les **positions initiales** et les **commandes de chaque drone** sont fournis par un **programme client en réseau** avec le simulateur.

### 4.2 Communication Réseau



#### 4.2.1 Utilisation des Sockets Java

- Utiliser les **Sockets TCP** de Java (`java.net.Socket`, `java.net.ServerSocket`)
- Le **simulateur** fonctionne en mode **serveur** (écoute sur un port)
- Le **programme client** envoie les informations au serveur

#### 4.2.2 Protocole de Communication

##### Phase 1 : Démarrage du Serveur

1. Lancer le simulateur avec l'option "serveur"
2. Le serveur écoute sur un port (ex: 5000)
3. Le serveur attend les informations du client

##### Phase 2 : Connexion du Client

1. Lancer le client
2. Le client se connecte au serveur (`localhost:5000`)
3. Le client envoie :
  - Taille de la carte (`width`, `height`)
  - Nombre de drones
  - Pour chaque drone : position (`x`, `y`), orientation (N/S/E/W), commandes

##### Phase 3 : Exécution de la Simulation

1. Le serveur reçoit les informations
2. Le serveur valide les données
3. Le serveur démarre la simulation (threads concurrents)
4. Le serveur affiche la simulation
5. Le serveur envoie au client les positions et orientations finales

##### Phase 4 : Affichage côté Client

1. Le client reçoit les positions finales
2. Le client affiche la carte avec les positions finales
3. Optionnel : afficher en continu les mouvements (streaming)

### 4.3 Structure du Protocole de Communication

#### Format des Données Échangées

##### Envoi Client → Serveur :

```
[MAP_WIDTH] [MAP_HEIGHT]
[NUM_DRONES]
[DRONE_1_X] [DRONE_1_Y] [DRONE_1_ORIENTATION] [DRONE_1_COMMANDS]
[DRONE_2_X] [DRONE_2_Y] [DRONE_2_ORIENTATION] [DRONE_2_COMMANDS]
...
```

##### Exemple :

```
20 20
2
0 5 N LMLMLMLMM
```

5 0 S MMRMMRMRRM

## Envoi Serveur → Client :

```
[SIMULATION_STATUS] (SUCCESS ou ERROR)
[FINAL_DRONE_1_X] [FINAL_DRONE_1_Y] [FINAL_DRONE_1_ORIENTATION]
[FINAL_DRONE_2_X] [FINAL_DRONE_2_Y] [FINAL_DRONE_2_ORIENTATION]
...
```

## Exemple :

```
SUCCESS
1 3 S
5 1 E
```

## 4.4 Architecture Client-Serveur

### 4.4.1 Côté Serveur

Package: com.drone.simulator.server

Classes principales:

```
— DroneServer (classe serveur)
  — startServer(port)
  — listenForConnections()
  — receiveConfiguration()
  — executeSimulation()
  — sendResults()
  — run()

— ServerConnectionHandler (implements Runnable)
  — handleClientConnection()
  — parseClientData()
  — validateData()
  — run()

— SimulatorMap (classe pour la carte)

— Drone (classe métier)

— DroneWorker (extends Thread)

— DisplayWorker (extends Thread)
```

## Exemple de Code Serveur :

```
public class DroneServer {
    private static final int PORT = 5000;

    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(PORT);
        System.out.println("Serveur démarré sur le port " + PORT);
        System.out.println("En attente de connexion client...");

        while (true) {
            Socket clientSocket = serverSocket.accept();
            System.out.println("Client connecté : " +
clientSocket.getInetAddress());
```

```

        ServerConnectionHandler handler =
            new ServerConnectionHandler(clientSocket);
        new Thread(handler).start();
    }
}

public class ServerConnectionHandler implements Runnable {
    private Socket socket;

    public ServerConnectionHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try (BufferedReader input = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
            PrintWriter output = new PrintWriter(
                new OutputStreamWriter(socket.getOutputStream()), true)) {

            // Recevoir la configuration
            String mapDimensions = input.readLine(); // "20 20"
            String[] dims = mapDimensions.split(" ");
            int width = Integer.parseInt(dims[0]);
            int height = Integer.parseInt(dims[1]);

            int numDrones = Integer.parseInt(input.readLine());
            List<Drone> drones = new ArrayList<>();

            for (int i = 0; i < numDrones; i++) {
                String droneLine = input.readLine();
                Drone drone = parseDroneData(droneLine);
                drones.add(drone);
            }

            // Exécuter la simulation
            SimulatorMap map = new SimulatorMap(width, height);
            executeSimulation(drones, map);

            // Envoyer les résultats
            output.println("SUCCESS");
            for (Drone drone : drones) {
                output.println(drone.getX() + " " +
                    drone.getY() + " " +
                    drone.getOrientation());
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

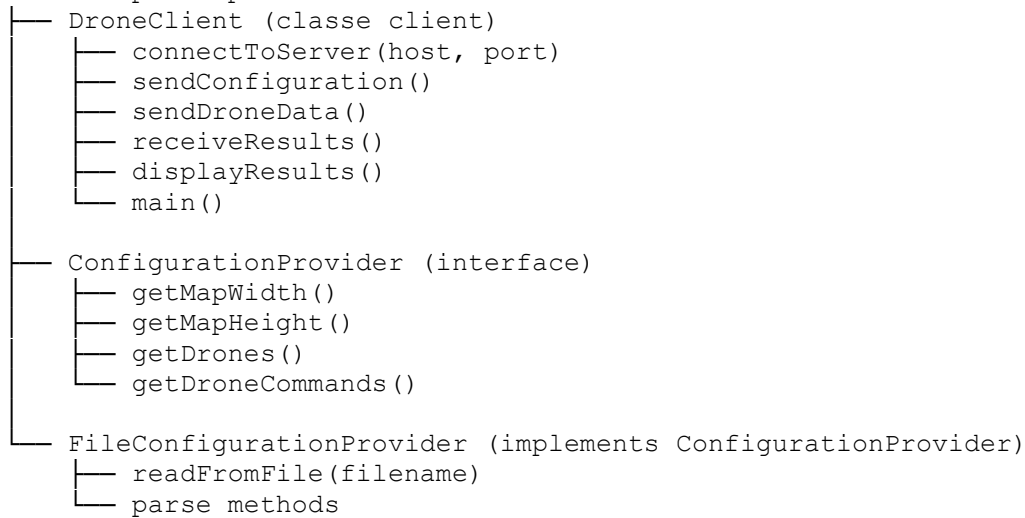
    private Drone parseDroneData(String line) {
        // Implémenter le parsing
        return null;
    }
}

```

#### 4.4.2 Côté Client

Package: com.drone.simulator.client

Classes principales:



### Exemple de Code Client :

```
public class DroneClient {
    private static final String HOST = "localhost";
    private static final int PORT = 5000;

    public static void main(String[] args) throws IOException {
        Socket socket = new Socket(HOST, PORT);
        System.out.println("Connecté au serveur");

        PrintWriter output = new PrintWriter(
            new OutputStreamWriter(socket.getOutputStream()), true);
        BufferedReader input = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));

        // Lire la configuration depuis un fichier
        ConfigurationProvider config =
            new FileConfigurationProvider("drones.txt");

        // Envoyer les dimensions de la carte
        output.println(config.getMapWidth() + " " + config.getMapHeight());
        output.flush();

        // Envoyer le nombre de drones
        List<Drone> drones = config.getDrones();
        output.println(drones.size());
        output.flush();

        // Envoyer les données de chaque drone
        for (Drone drone : drones) {
            String droneData = drone.getX() + " " +
                drone.getY() + " " +
                drone.getOrientation() + " " +
                drone.getCommands();
            output.println(droneData);
            output.flush();
        }

        // Recevoir les résultats
        String status = input.readLine();
        if ("SUCCESS".equals(status)) {
```

```

        System.out.println("Simulation réussie !");

        for (int i = 0; i < drones.size(); i++) {
            String resultLine = input.readLine();
            String[] parts = resultLine.split(" ");
            int x = Integer.parseInt(parts[0]);
            int y = Integer.parseInt(parts[1]);
            char orientation = parts[2].charAt(0);

            System.out.println("Drone " + i + " : " +
                               x + " " + y + " " + orientation);
        }

        socket.close();
    }
}

```

## 4.5 Version Communicante

### 4.5.1 Programmation du Client

- Créer un programme client capable de :
  - Se connecter au serveur (utilisation de Socket)
  - Lire la configuration depuis un fichier texte
  - Envoyer au serveur : taille carte, positions, orientations et commandes des drones
  - Parser les réponses du serveur
  - Afficher les positions finales

### 4.5.2 Réception et Exécution côté Serveur

- Créer un serveur capable de :
  - Écouter les connexions entrantes
  - Recevoir la configuration du client
  - Valider les données reçues
  - Exécuter la simulation concurrente
  - Envoyer les résultats au client

### 4.5.3 Communication des Résultats et Affichage

- Implémenter le streaming des résultats :
  - Le serveur envoie les positions et orientations finales
  - Le client reçoit et affiche les résultats
  - Optionnel : affichage en continu des mouvements pendant la simulation

## 4.6 Points Importants pour la Version Communicante

- **Gestion des erreurs réseau** : Implémenter la gestion appropriée des exceptions `IOException`
- **Timeouts** : Ajouter des timeouts pour éviter que le client n'attende indéfiniment
- **Fermeture des ressources** : Utiliser les try-with-resources pour fermer proprement les connexions
- **Multi-client** : Le serveur devrait pouvoir gérer plusieurs clients (utiliser un thread par client)

- **Format de protocole** : Définir un protocole clair et robuste (considérer le JSON ou un format délimité)
- 

## 5. Conseils d'Implémentation

### 5.1 Gestion des Obstacles

```
public class SimulatorMap {
    private char[][] grid;
    private int width;
    private int height;
    private static final char OBSTACLE = '#';
    private static final char EMPTY = '.';

    public boolean isBlocked(int x, int y) {
        if (x < 0 || x >= width || y < 0 || y >= height) {
            return false; // Wrapping
        }
        return grid[y][x] == OBSTACLE;
    }

    public int wrapCoordinate(int coord, int max) {
        return ((coord % max) + max) % max;
    }
}
```

### 5.2 Classe Drone

```
public class Drone {
    private int x, y;
    private char orientation; // N, S, E, W
    private String commands;

    public void executeCommand(char cmd, SimulatorMap map) {
        switch (cmd) {
            case 'L':
                turnLeft();
                break;
            case 'R':
                turnRight();
                break;
            case 'M':
                moveForward(map);
                break;
            case 'B':
                moveBackward(map);
                break;
        }
    }

    private void moveForward(SimulatorMap map) {
        int newX = x, newY = y;
        switch (orientation) {
            case 'N': newY++; break;
            case 'S': newY--; break;
        }
    }
}
```

```

        case 'E': newX++; break;
        case 'W': newX--; break;
    }

    // Wrapping
    newX = map.wrapCoordinate(newX, map.getWidth());
    newY = map.wrapCoordinate(newY, map.getHeight());

    if (!map.isBlocked(newX, newY)) {
        x = newX;
        y = newY;
    }
}
}

```

## 5.3 Utilisation de Collections Thread-Safe

Pour la version concurrente, utiliser des collections synchronisées :

```

// ✗ Non-sécurisé
List<Drone> drones = new ArrayList<>();

// ✔ Sécurisé pour la concurrence
List<Drone> drones = Collections.synchronizedList(new ArrayList<>());

```

## 5.4 Format de Fichier pour le Test

**Fichier de configuration** (drones.txt) :

```

20 20
2
0 5 N LMLMLMLMM
5 0 S MMRMMRMRRM

```

---

## 6. Évaluation et Remise

### 6.1 Critères de Notation

- **Fonctionnalité** : Le programme répond-il aux exigences ?
- **Qualité du code** : Lisibilité, commentaires, structure
- **Gestion des erreurs** : Exceptions traitées correctement
- **Synchronisation** : Pas de race conditions
- **Respect des consignes** : Suivre la structure proposée

### 6.2 Remise

- **Format** : Archive .zip contenant le projet Maven complet
  - **Documentation** : README.md expliquant comment compiler et exécuter
  - **Tests** : Inclure des fichiers de test pour valider chaque version
-

Bon courage à tous